Improved Algorithm of RRT Path Planning and the Application in Complex Environment

Le Chang(✉), Dongyang Zhang, Yueling Dai, and Chao Yang

Beijing Aerospace Times Laser Inertial Technology Company, Ltd., Beijing 100094, China
cl_buaa@126.com

Abstract. This paper proposes an improvement to the RRT algorithm for path planning. The new bi-directional RRT utilizes comparison optimization and fuzzy inference to address the limitations of the base algorithm, such as instability and potential deviation from the optimal path. Firstly, two random trees are established separately using the starting position and target position as the root nodes. Two trees are extended to the nodes of their opposite trees, according to the introduced effort function simultaneously. Secondly, a more reasonable measure function and a comparison optimization strategy are introduced to greatly increase the planning stability. Finally, exploring probability and the extension step length are adjusted by fuzzy control theory dynamically combined with environment information of current node, so that the efficiency and ability to search unknown areas of the algorithm are improved effectively. Simulations confirm the algorithm's enhanced stability in complex environments, allowing for robust path planning. Furthermore, the achieved paths demonstrate a high degree of optimality, minimizing unnecessary movement.

Keywords: Rapidly-Exploring Random Trees · Path Planning · Comparison Optimization · Fuzzy Inference

1 Introduction

The autonomous mobile robot is the research hotspot in recent years, and path planning of robot is an important issue [1]. The traditional path planning algorithms, such as template matching algorithm, genetic algorithm, A* algorithm, artificial potential field algorithm and so on, have their own limitations [2, 3]. Different from the traditional methods, the Rapidly-Exploring Random Trees (RRT) algorithm finds a path between the origin and destination by sampling randomly in the state space and searching in the global zone [4]. The main advantage of this algorithm is that, the search space are not required, and can quickly find a feasible path [5].

However, the basic RRT algorithm also has a series of problems in practice. (1) Random sampling in global scope, makes the search tree expansion has blindness and may result in the reduction of convergence speed [6]. (2) The deviation of multiple search results is large in the same environment because of the randomness, which means the poor stability [7]. (3) The generated path may be not smooth enough to be executed by

L. Yan et al. (Eds.): ICGNC 2024, LNEE 1345, pp. 177–188, 2025.
https://doi.org/10.1007/978-981-96-2232-0_19

the mobile robot [8]. Aiming at the shortages of the basic RRT algorithm, many scholars put forward a lot of improvements. Wang Kun [9] proposed a DRRT-Connect algorithm that can generate four trees simultaneously and has an adaptive step size adjustment during expansion, which significantly improves the efficiency of path planning. However, the method of adaptive step size adjustment is slightly simple, and the final path length is almost not improved. Huang Yifan [10] proposed an improved RRT-Connect algorithm, which introduces the reselection of parent nodes considering ancestor nodes, designs dynamic step size optimization strategies, to reduce the convergence time of the algorithm, but there are still many redundant nodes and turning points on the path. Du Chuansheng [11] proposed a greedy RRT algorithm based on double-direction expansion of the same root, which adds a force field in the expansion of new nodes and prunes the path to reduce the path length and the number of nodes.

2 Basic RRT Algorithm

The Rapidly-exploring Random Tree (RRT) is depicted in Fig. 1. The method sets the initial point as a root node, and attempts to extend the current random tree toward target point iteratively. At last, an effective path can be found reversely from the random tree which has been established.

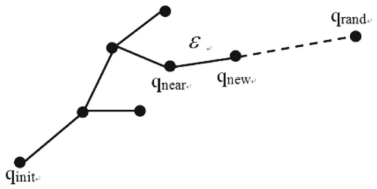


Fig. 1. An RRT extension

The Bidirectional RRT algorithm tackles the limitations of the basic RRT by simultaneously building two random trees. One tree starts from the initial position, and the other from the target. Both trees utilize the same extension method as the basic RRT to explore the environment. This concurrent exploration from both ends allows for faster convergence. If a node from one tree gets sufficiently close to a node from the other, a connection is established. Finally, the algorithm searches for a valid path from the connection point back to the start and goal points using the respective trees.

3 Bidirectional RRT Based on the Cost Function

Bidirectional RRT algorithm takes the last node of one tree as the target point to extend another random tree. However, if the leaf node, which is used as the target, deviates greatly from the direction of random tree to be extended, this algorithm may produce excess branches and the optimal extending point is not easy to find. As a result, a cost function is introduced to optimize the selection of the target point.

Record the path cost $h(q_t)$ between the current node and the start node when extending the random trees.

$$h(q_t) = Nnum \times \varepsilon \tag{1}$$

Nnum represents the node amount growing from the starting node to the current node. Calculate the cost function $\varphi(q_t)$ of each node in another tree when choosing the target point.

$$\varphi(q_t) = f(q_t) + h(q_t) \tag{2}$$

In Eq. (2), $f(q_t)$ stands for Euclidean distance information.

$$\mathrm{f}(q_t) = \sqrt{[q_e(x) - q_t(x)]^2 + [q_e(y) - q_t(y)]^2} \tag{3}$$

where, q_e is the end node of current tree, locating in $(q_e(x),\ q_e(y))$. q_t is the node of another tree.

Extend the tree to the node that has the minimum $\varphi(x_t)$ in another tree, as the inspiring information can better guide the growth of random trees.

4 Bidirectional RRT Added with Comparison Optimization

The basic RRT algorithm always chooses the nearest node from q_{rand} as q_{new} to extend, and it does not take full advantages of known information to guide the expansion of random tree. So, the differences among the paths of every test are large.

In this section, a more reasonable metric function, which contains the distance and angle information of node, is introduced to optimize the expansion of random tree nodes. q_{new} is chosen by means of comparison optimization.

Steps are shown as below (taking one tree for example);

1) Find k points, $(q_{near1}, q_{near2}, \ldots\ldots, q_{neark})$, who have the nearest Euclidean distances to q_{rand}. Take k as 5 here.
2) Get temporary extension nodes $(q_{temp1}, q_{temp2}, \ldots\ldots, q_{tempk})$ by single step extension, and calculate the cost functions $\phi(q_i)$ of these nodes,

$$\phi(q_i) = \lambda_1 \times L(q_i) + \lambda_2 \times A(q_i) \tag{4}$$

where $\mathrm{L}(q_i)$ represents the distance information of q_{tempi}.

$$\begin{cases} L(q_i) = N_1(Dis(i)) \\ Dis(i) = g(q_i) + h(q_i) \end{cases} \tag{5}$$

$\mathrm{g}(q_i)$ is the diagonal distance between q_{tempi} and target point. .

$$\begin{aligned} g(q_i) &= max(|q_{tempi}(x) - q_{goal}(x)|, |q_{tempi}(y) - q_{goal}(y)|) \\ &+(\sqrt{2} - 1)min(|q_{tempi}(x) - q_{goal}(x)|, |q_{tempi}(y) - q_{goal}(y)|) \end{aligned} \tag{6}$$

$h(q_i)$ is the path length between q_{tempi} and start node, which can be obtained by the same way of h(q) in Sect. 2. It is clear from the equations above that, the smaller Dis(i) is, the less cost of path will be, and the path will be closer to the optimal result.

$A(q_i)$ represents the angle information of q_{tempi}.

$$\begin{cases} A(q_i) = N_2(\theta(i)) \\ \theta(i) = arc(\text{Cosine}(i)),\ (0 < \theta(i) < 90) \end{cases} \tag{7}$$

where $\theta(i)$ is the angle between $\overrightarrow{q_{otempi}}$ and $\overrightarrow{q_{orand}}$, shown as Fig. 2. Cosine(i) can be obtained by cosine formula. The path is more gentle if $\theta(i)$ is smaller.

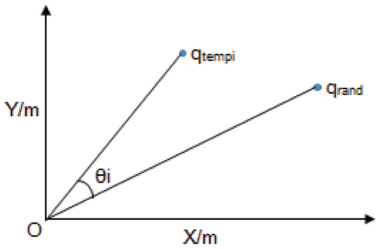


Fig. 2. Diagram of angle θ

In order to unify two different sources of data (distance and angle) into the same frame of reference, these two variables are normalized by linear functions. $N_1(Dis(i))$ and $N_2(\theta(i))$ are shown below.

$$\begin{cases} N_1(Dis(i)) = \frac{Dis(i) - Dis_{\min}}{Dis_{\max} - Dis_{\min}}, & Dis_{\min} = \min(Dis(i)) \\ & Dis_{\max} = \max(Dis(i)) \\ N_2(\theta(i)) = \frac{\theta(i) - \theta_{\min}}{\theta_{\max} - \theta_{\min}}, & \theta_{\min} = \min(\theta(i)) \\ & \theta_{\max} = \max(\theta(i)) \end{cases} \tag{8}$$

λ_1 and λ_2 are the corresponding weights. This algorithm is abbreviated as BC-RRT algorithm.

5 The Improved Path Planning Algorithm BCF-RRT

5.1 Parameters Analysis

During analysis, the target node is chosen as the random node with high probability when P_g is larger. This will lead to a single selection of random nodes, so that the ability of exploring the environment becomes weak. As the result, the algorithm needs more time to bypass obstacles and get a safe collision-free path. On the other hand, the total number of nodes and the average path length are gradually reduced with the increasing of P_g. The number of nodes and running time decrease, the resulting path length increases, and the simulation time decreases when the step increases. This indicates that if the step ε is too large, the algorithm will be over sensitive to the environment and the ability to bypass obstacles is reduced, especially for the environment with dense obstacles.

5.2 Detailed Process of the Fuzzy Inference

In order to optimize the value of P_g and ε, the fuzzy inference based on environment information is added to BC-RRT algorithm. P_g and ε are updated continuously to adapt to the current environment. Choose larger P_g and ε to enhance the direction of random tree growth and speed up the planning in simple environment. Choose smaller P_g and ε to enhance the ability of searching and find a feasible path to avoid obstacles in complex environment.

At the beginning of the algorithm, initialize P_{g1}, P_{g2}, ε_1, ε_2, which are the initial values of exploring probability and extension step lengths of two trees. Set $P_{g1} = 0.5$, $P_{g2} = 0.5$. The values of ε_1 and ε_2 are determined by the size of the space. P_{g1}, P_{g2}, ε_1, ε_2 are updated by fuzzy inference according to the environment information of current node every cycle.

N-ratio is the ratio of the number of obstacles within a certain range of current node and the total number of obstacles. L-ratio is the ratio of current extension step length ε_i and the average distance between current node and those obstacles within a certain range. Step-ratio is the ratio of ε_i and the initial step length. P_g is the exploring probability. The domain and fuzzy division number of each variable is designed as follows.

$$\text{N-ratio} \in [0,0.15,0.3],\ \text{T(N-ratio)} \in \{\text{PS,PM,PB}\}\ ;$$
$$\text{L-ratio} \in [0,1.5,3],\ \text{T(L-ratio)} \in \{\text{PS,PM,PB}\}\ ;$$
$$P_g \in [0.3,0.5,0.7],\ \text{T}(P_g) \in \{\text{PS,PM,PB}\}\ ;$$
$$\text{Step-ratio} \in [0.5,1,1.5],\ \text{T(Step-ratio)} \in \{\text{PS,PM,PB}\}\ ;$$

Where PS, PM, PB means small, middle and big respectively.
The membership functions of each variable are shown in Fig. 3.

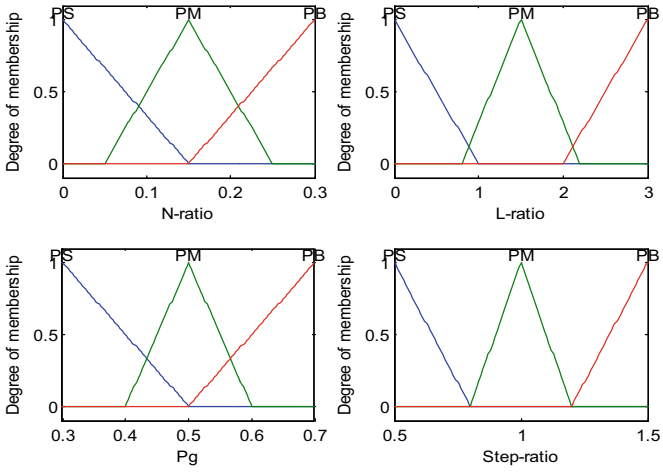


Fig. 3. Membership functions of each variable.

Design the following nine fuzzy rules:

1) If(N-ratio is PS)and (L-ratio is PS)then (P_g is PM)(Step-ratio is PM).
2) If (N-ratio is PS)and (L-ratio is PM)then(P_g is PB)(Step-ratio is PB).
3) If (N-ratio is PS)and (L-ratio is PB)then(P_g is PB)(Step-ratio is PB).
4) If(N-ratio is PM)and(L-ratio is PS)then (P_g is PS)(Step-ratio is PS).
5) If(N-ratio is PM)and (L-ratio is PM)then (P_g is PM)(Step-ratio is PM).
6) If (N-ratio is PM)and (L-ratio is PB)then (P_g is PB)(Step-ratio is PB).
7) If (N-ratio is PB)and (L-ratio is PS)then (P_g is PS)(Step-ratio is PS).
8) If(N-ratio is PB)and (L-ratio is PM)then (P_g is PS)(Step-ratio is PS).
9) If(N-ratio is PB)and (L-ratio is PB)then (P_g is PM)(Step-ratio is PM).

In the end, average maximum membership degree method is adopted to deblur.

6 Path Smoothing and Pruning of Redundant Nodes

Redundant nodes in the path generated by BCF-RRT algorithm are inevitable, so it is need to prune extra nodes and smooth the path. The detailed process is listed as follows.

(1) *Establish a new path T_{new}, which uses the first node of the path T generated by BCF-RRT as the root node.*
(2) *Link the current node of T_{new} and T's following nodes by turn, until it meets an obstacle.*
(3) *Add the nearest node whose connection with current node of T doesn't meet the obstacles to T_{new}, and use it as the current node of T_{new}.*
(4) *Repeat Step (2) and (3) until meeting the target.*
(5) *Connect nodes from the target reversely to get a new path T_{new}.*
(6) *Gets a smooth path by B-spline smoothing function.*

7 Simulations and Analyses

To verify the effectiveness of the BCF-RRT algorithm, taking the dot robot as a particle, do simulations in different complex environment, and compare the result with basic RRT. The simulation was done with Matlab on a PC with Intel Celeron 3.3 GHz CPU.

The map used in the simulation is a rectangle with [1000*1000]. N round obstacles are randomly placed in this region. The radiuses of obstacles are randomly set as well. The coordinate of the starting position is (0,0), and the target position is (1000, 1000). The simulation is to plan a valid path, with the limitation of one meter between the robot and the obstacle.

7.1 Simulations in Environment I

Figure 4 is a map randomly created by programs, and the number of obstacles is 50. Easy to find that, around the starting point, there are more obstacles, and the obstacles are concentrated. On the other hand, fewer obstacles are dispersed near the target point.

Figure 5 is a path planned by the basic RRT algorithm, and the random tree created by it. Blue line in Fig. 5(b) is the un-pruned path, and the red line is the pruned path. Parameters in RRT are selected as follows. $p_g = 0.5$ and $\varepsilon = 30$. Figure 6 is the path

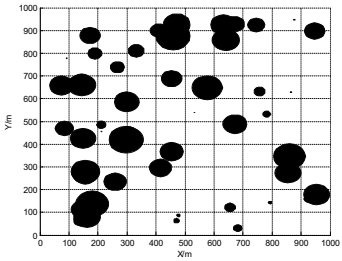
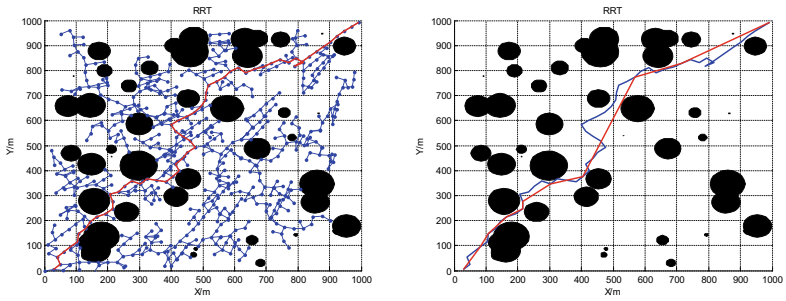
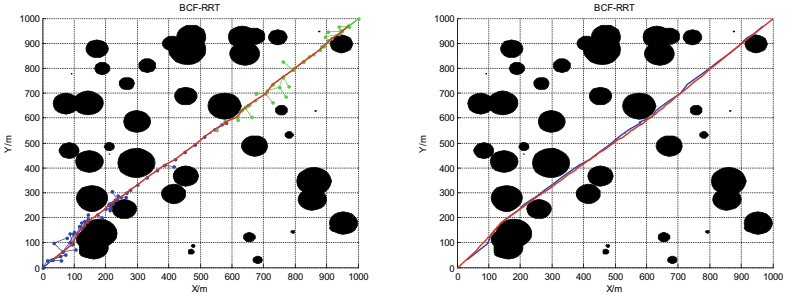


Fig. 4. The figure of test map 1



(a)The original path and random tree (b)The original path and path after pruning

Fig. 5. The result figure of RRT path planning in map 1



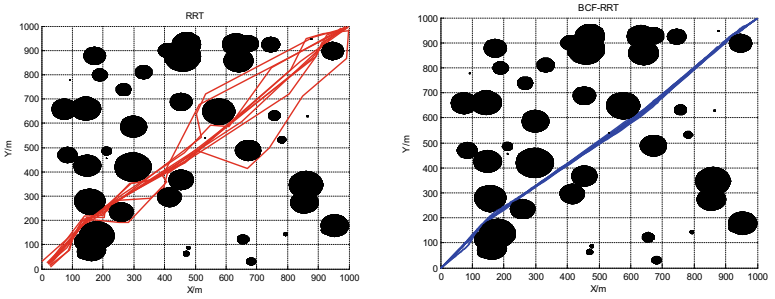
(a)The original path and random trees (b)The original path and path after pruning

Fig. 6. The result figure of BCF-RRT path planning in map 1

planned by BCF-RRT and its random tree. In Fig. 6(a), blue and green lines represent the two trees growing from the starting point and target point separately. In Fig. 6(b), the blue line is the un-pruned path, and the red line represents the pruned path. Parameters in BCF-RRT are selected as follows. $p_{g1} = 0.5, p_{g2} = 0.5$, $\varepsilon_1 = 30, \varepsilon_2 = 30$, $\lambda_1 = 0.6, \lambda_2 = 0.4$.It is easy to find that, the tree of traditional RRT algorithm(Fig. 5) extends with more randomness, and the expansion of nodes is not related to the distribution of obstacles. BCF-RRT algorithm (Fig. 6) considers environment differences and

comparison optimizations on the contrary. In the environment with fewer obstacles, the random tree is extended directly towards the target with fewer branches and larger steps; in the environment with more and concentrated obstacles, the random tree is extended to each direction with relatively smaller steps to bypass obstacles.

To validate the stability and effectiveness of BCF-RRT algorithm, run the basic RRT and BCF-RRT in the map 1 for 10 times.



(a)RRT (b) BCF-RRT

Fig. 7. The result figure after running 10 times in map 1

Figure 7 shows the pruned paths of basic RRT and BCF-RRT in map 1. Figure 8 shows the pruned paths of BCF-RRT in map1. The number of random extended points, running time, length of original path and final path are listed in Table 1.

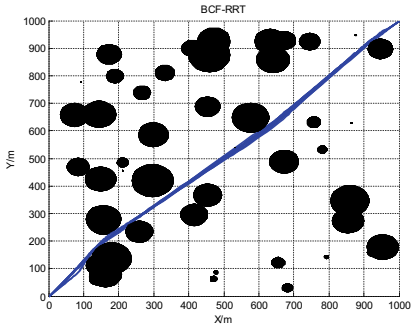


Fig. 8. The result figure of BCF-RRT after running 10 times in map 1

To clearly test the enhancements of BCF-RRT, compare the un-pruned length of the tree with that in other three algorithms (basic RRT, bidirectional RRT with cost functions, bidirectional RRT with comparison optimization, and the BCF-RRT) in map I.

Figure 9 is the comparison of average original path length after running 50 times among RRT, B-RRT, BC-RRT, BCF-RRT in map I.

The comparison of average length and standard deviation among four algorithms is listed in Table 2.

Table 1. Property parameter table in map 1

	RRT				BCF-RRT			
	Nrand	Time(s)	Length(m)	Length_C(m)	Nrand	Time(s)	Length(m)	Length_C(m)
1	664	0.6796	1760.1	1620.4	94	0.1434	1520.0	1462.1
2	807	0.5542	1917.5	1683.8	79	0.1037	1492.3	1459.0
3	491	0.3599	1790.7	1642.7	72	0.0990	1502.9	1460.0
4	134	0.0452	1583.3	1567.6	79	0.2114	1502.3	1458.3
5	212	0.0878	1646.6	1528.4	82	0.1233	1503.7	1460.8
6	491	0.2151	1881.8	1673.8	69	0.1911	1489.7	1461.4
7	1404	2.1248	1924.5	1611.4	50	0.0886	1501.7	1458.2
8	585	0.2713	1524.9	1517.4	69	0.1092	1479.6	1459.7
9	500	0.2294	1701.6	1595.5	75	0.1242	1501.3	1460.4
10	1107	1.1680	1974.9	1737.8	65	0.1308	1487.8	1460.3
average value	639	0.5735	1770.6	1617.9	73	0.1325	1498.1	1460.0
standard deviation	385	0.6386	154.7291	69.6416	11	0.0399	11.1922	1.2682

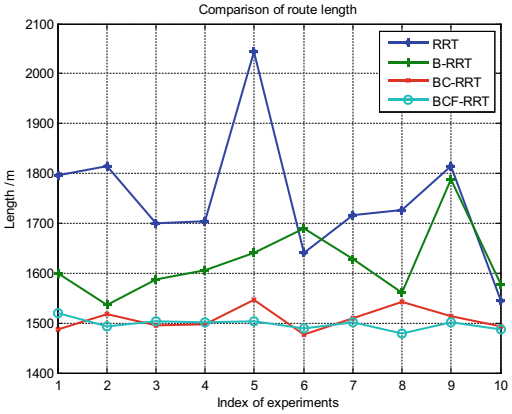


Fig. 9. The path length figure of different algorithms in map1

It can be clearly found from the result that, basic RRT algorithm has great randomness, and the planned path can be easily far from the optimal result; B-RRT can reduce the total length compared with basic RRT, but its path also has significant randomness. BC-RRT can reduce the difference between each path, for the more proper measurement functions. Finally, the BCF-RRT gets the shortest average length of original paths, and has the minimum standard deviation. Meanwhile, the average running time is greatly increased.

Table 2. Performance contrast of different path planning algorithm in map 1

	Average length (m)	Standard deviation
RRT	1773.2	155.2783
B-RRT	1620.9	72.4735
BC-RRT	1512.6	22.5091
BCF-RRT	1497.9	11.2322

7.2 Simulations in Environment II

Figure 10 is the second map randomly created by programs. In this figure, the number of obstacles is 100, and obstacles distribute densely in the whole area. So it is more difficult to plan a path compared with Environment I. Similarly, run basic RRT and BCF-RRT in map 2 for 10 times separately, set the same parameters as Sect. 7.1, and obtain the results as in Fig. 11.

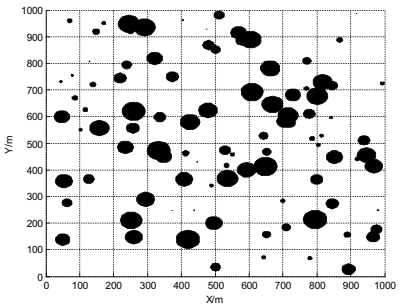
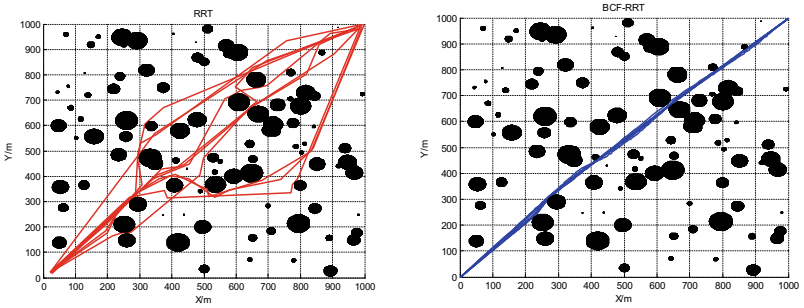


Fig. 10. The figure of test map 2



(a)RRT (b) BCF-RRT

Fig. 11. The result figure after running 10 times in map 2

The numbers of random extended nodes, running time, length of original/pruned path of RRT and BCF-RRT are separately listed as Table 3.

Table 3. Property parameter table in map2

	RRT				BCF-RRT			
	Nrand	Time(s)	Length(m)	Length_C(m)	Nrand	Time(s)	Length(m)	Length_C(m)
1	451	0.3103	1890.5	1646.7	166	0.3731	1511.9	1485.5
2	581	0.4112	2020.1	1799.1	186	0.4009	1535.4	1488.5
3	1842	6.0473	1730.5	1614.7	143	0.3563	1517.5	1486.5
4	306	0.2053	1573.1	1497.7	142	0.3170	1518.5	1485.7
5	493	0.3573	1716.2	1611.2	134	0.3552	1509.4	1485.9
6	830	0.8189	1650.5	1500.4	175	0.3938	1512.6	1485.6
7	541	0.3948	1818.0	1527.9	195	0.4602	1535.6	1488.1
8	1693	4.0000	1811.8	1540.1	106	0.2237	1510.8	1481.2
9	432	1.8918	1891.8	1635.5	122	0.3440	1514.2	1485.4
10	747	0.6674	1742.2	1577.1	155	0.4159	1499.6	1486.2
average value	791	1.5104	1784.5	1595.0	152	0.3640	1516.6	1486.9
standard deviation	537	1.9732	130.0172	89.9838	28	0.0639	12.3386	1.8698

Figure 12 is the comparison of average original path length after running 50 times among RRT, B-RRT, BC-RRT, BCF-RRT in map II.

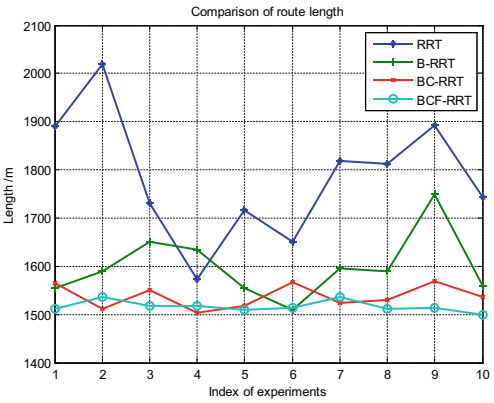


Fig. 12. The path length figure of different algorithm in map2

The comparison of average length and standard deviation among four algorithms is listed in Table 4.

It can be concluded from the simulation results, in the environment with dense obstacles, the efficiency and stability of basic RRT will be both decreased. However, the BCF-RRT algorithm can improve the performance in both efficiency and stability, and it can better approach the optimal result than basic RRT.

Table 4. Performance contrast of different path planning algorithm in map2

	Average length(m)	Standard deviation
RRT	1785.4	132.1025
B-RRT	1598.4	66.3842
BC-RRT	1537.1	23.1978
BCF-RRT	1516.1	12.3244

8 Conclusions

Aiming at the path planning of mobile robot in complex environment, an improved bidirectional RRT algorithm is proposed. During the path planning, environmental constraints, distance constraints and angles constraints are introduced, which greatly reduced the randomness of basic RRT algorithm. Simulations demonstrate that, in different environment, the improved BCF-RRT algorithm significantly enhances the stability of the path planning of RRT, and it is benefit to approach the optimal result. The BCF-RRT algorithm ensures the smoothness, stability, and collision-free ability of the path, while improving the search efficiency in complex environment.

References

1. Karthik, K., Nitin, S., Chinmay, D.: A survey of path planning algorithms for mobile robots. Vehicles **3**(3), 448–468 (2021)
2. Song, X.R., Reng, Y., Gao, S.: Survey on technology of mobile robot path planning. Comput. Meas. Control **27**(4), 1–5 (2019)
3. Chiang, H.T.L., Hsu, J., Fiser, M.: Dynamic motion planning via learning reachability estimators from RL policies. IEEE Robot. Autom. Lett. **4**(4), 4298–4305 (2019)
4. Li, X., Tong, Y.: Path planning of a mobile robot based on the improved RRT algorithm. Appl. Sci. **14**(1), 25–28 (2023)
5. Zhao, C.I., Ma, X., Zhang, C.T.: RRT-connect path planning algorithm based on gravitational field guidance. Electron. Meas. Technol. **44**(22), 44–49 (2021)
6. Szhang, L., Shi, X., Yi, Y.: Mobile robot path planning algorithm based on RRT_Connect. Electronics **28**(9), 25–29 (2023)
7. Zhang, X., Zhu, T., Xu, Y.: Local path planning of the autonomous vehicle based on adaptive improved RRT algorithm in certain lane environments. Actuators **11**(4), 109–117 (2022)
8. Li, Y., Ma, S.: Navigation of apple tree pruning robot based on improved RRT-connect algorithm. Agriculture **13**(8), 1495–1499 (2023)
9. Wang, K., Huang, B., Zeng, G.H.: Faster path planning based on improved RRT-connect algorithm. J. Wuhan Univ. (Nat. Sci. Ed.) **65**(3), 283–289 (2019)
10. Huang, Y.F., Hu, L.K., Xue, W.C.: Mobile robot path planning based on improved RRT-connect algorithm. Comput. Eng. **47**(8), 22–28 (2021)
11. Du, C.S., Gao, H.B., Hou, Y.X.: Greedy RRT path planning algorithm with same root bidirectional extension. Comput. Eng. Appl. **59**(21), 312–318 (2023)